

(a) affine: leaf, no stack frame

Value	Register	Actual instruction
x	fa0	<code>fmul.s fa0, fa0, fa1</code>
scale	fa1	(same <code>fmul.s</code>)
offset	a0	<code>fcvt.s.w ft0, a0</code>
return	fa0	<code>fadd.s fa0, fa0, ft0; ret</code>

Live across call affine:

`s0 = array    s1 = n    s2 = i`
`fs0 = scale    fs1 = sum`

Prologue slots save the caller's old registers;
current loop values stay in `s*/fs*` across affine.

(b) sum_positive: 64-byte frame

old sp = new sp + 64 (high)

+56	saved ra (sd)
+48	saved s0 (sd)
+40	saved s1 (sd)
+32	saved s2 (sd)
+24	saved fs0 (fsd)
+16	saved fs1 (fsd)
+08	unused
+00	unused

new sp + 0 (low)

64 bytes = 4 x 16; no local/spill slots